



Security Audit

Umbrae (DeFi)

Table of Contents

Executive Summary	4
Project Context	4
Audit Scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	14
Audit Resources	14
Dependencies	14
Severity Definitions	15
Status Definitions	16
Audit Findings	17
Centralisation	62
Conclusion	63
Our Methodology	64
Disclaimers	66
About Hashlock	67

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

The Umbrae team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

Umbrae is launching the first multi-chain DLMM on Base Network with the lowest operation fees in DeFi. The platform delivers 97-99% transaction success rates through proprietary infrastructure with ongoing security partnership through Hashlock. Public users access DLMM trading, MEV protection, and credit-based AI chat with API integration across Discord and Telegram. Ignis Elite members receive exclusive access to automated 5-agent trading, 30% fee distribution, white-glove support, and priority features across Solana, Base, Arbitrum, and expanding chains.

Project Name: Umbrae

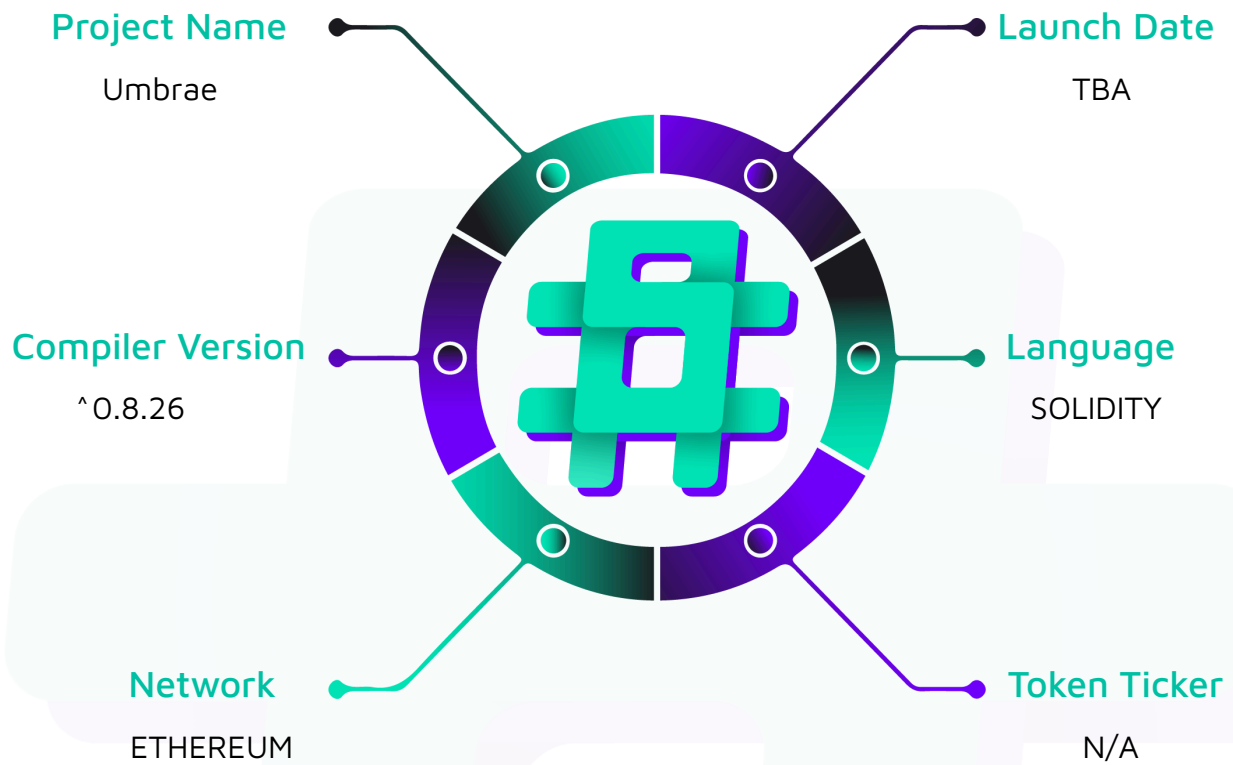
Project Type: Defi

Compiler Version: ^0.8.26

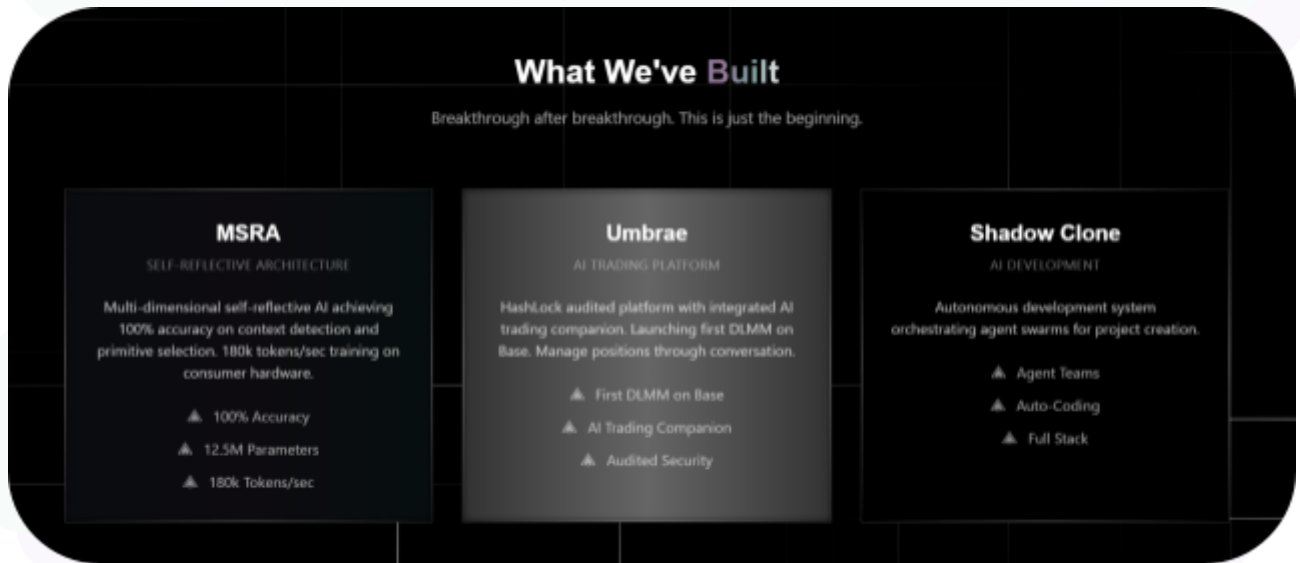
Website: <https://www.ignislabs.ai/>

Logo:

U M Ɓ ɹ A ≡

Visualised Context:

Project Visuals:



Audit Scope

We at Hashlock audited the solidity code within the Umbrae project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	Umbrae Smart Contracts
Platform	Ethereum / Solidity
Audit Date	February, 2026
Scope	Ignis-AI-Labs/dlmm/tree/master/src
Audited GitHub Commit Hash	e1e113f5bbc5f10b3299f0820507e409f0cd4de6
Fix Review GitHub Commit Hash	38a7b60b59b37885f278ecc8d771731724f911e5

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts.



The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the [Audit Findings](#) section. The list of audited assets is presented in the [Audit Scope](#) section and the project's contract functionality is presented in the [Intended Smart Contract Functions](#) section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

3 High severity vulnerabilities

7 Medium severity vulnerabilities

7 Low severity vulnerabilities

6 QA

Caution: Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
UmbraeTimelock.sol Governance timelock for protocol operations - Controls factory and router upgrades - Proposer/executor roles for multisig	Contract achieves this functionality.
ILCalculator.sol Library for calculating impermanent loss (IL) for LP positions - Calculates IL percentage based on price movements - Provides position health scoring	Contract achieves this functionality.
PositionAnalyzer.sol Helper contract for analyzing DLMM liquidity positions - Analyzes position status - Provides recommended bin ranges around market price	Contract achieves this functionality.
UniswapV3PriceOracle.sol Fetches TWAP prices from Uniswap V3 pools	Contract achieves this functionality.
UmbraeLBFactoryV2.sol Factory for creating and managing upgradeable pairs - Deploys BeaconProxy pairs (upgradeable via beacon) - Permissionless pair creation with token verification system - Per-pair pause functionality - Protocol fee recipient management	Contract achieves this functionality.

UmbraeLBPairBeacon.sol Beacon contract for LBPair proxies - All pairs share same beacon for atomic upgrades - Schedule/execute/cancel upgrade flow with delay - Emergency pause mechanism (instant global pause)	Contract achieves this functionality.
UmbraeLBPairPaused.sol Emergency paused implementation that reverts all calls - Used during protocol emergencies to instantly halt all pairs - Activated via beacon.emergencyPause()	Contract achieves this functionality.
UmbraeLBPairUpgradeable.sol Main liquidity pair contract implementing DLMM - Swap tokens across discrete price bins - Add/remove liquidity to bins - Flash loans with fee - Dynamic fees based on volatility - TWAP oracle for price history - Per-bin fee tracking with residual accumulation	Contract achieves this functionality.
UmbraeLBRouterUpgradeable.sol UUPS upgradeable router for token swaps and liquidity operations - Multi-hop swaps with accurate quotes - Add/remove liquidity with slippage protection - Price oracle integration - Distribution validation - Active bin slippage protection	Contract achieves this functionality.
UmbraeLBTokenUpgradeable.sol ERC-1155-like token for LP positions. Tracks LP token	Contract achieves this functionality.

balances per bin ID.	
BinHelper.sol Helper for bin price calculations - Converts between bin IDs and prices	Contract achieves this functionality.
BinManager.sol Library for managing bin state operations - Add/remove liquidity from bins - Execute swaps within bins - Check bin composition and liquidity - Validate bin state consistency	Contract achieves this functionality.
BitMath.sol Bit manipulation operations (adapted from Uniswap V3) - Most/least significant bit calculations - Used for O(1) bitmap bin searches	Contract achieves this functionality.
FeeCalculator.sol Pure functions for fee calculations - Calculates swap fees (base + variable based on volatility) - Volatility decay over time - Fee parameter validation	Contract achieves this functionality.
FeeHelper.sol Helper for fee calculations and distributions - Default fee parameters - Dynamic fee calculation with volatility - Protocol/LP fee distribution - Volatility decay with filter period protection	Contract achieves this functionality.
FlashLoanHelper.sol Helper library for flash loan operations - Validates flash loan parameters	Contract achieves this functionality.

- Calculates fees and expected repayments - Verifies repayment amounts - Builds callback data	
FullMath.sol Helper library for 512-bit math for overflow-safe multiplication/division. Adapted from Uniswap V3	Contract achieves this functionality.
LiquidityCalculator.sol Pure functions for liquidity calculations - Calculates LP shares for adding liquidity - Determines token amounts based on bin position - Calculates removal amounts - Validates liquidity parameters	Contract achieves this functionality.
OracleHelper.sol Helper for managing price oracles - TWAP calculations over time windows - Volatility averaging - Sample lookup by timestamp	Contract achieves this functionality.
OracleLibrary.sol Library for oracle operations in DLMM pairs - Wraps OracleHelper for pair integration - Checks if oracle should update - Validates oracle parameters	Contract achieves this functionality.
ReentrancyGuard.sol Prevents reentrancy attacks using EIP-1153 transient storage	Contract achieves this functionality.
SafeCast.sol Safe type casting with overflow checks	Contract achieves this functionality.
SwapCalculator.sol Pure functions for swap calculations	Contract achieves this functionality.

- | | |
|---|--|
| <ul style="list-style-type: none">- Calculates swap output within a bin- Multi-bin swap calculations- Price impact calculations- Required input for desired output | |
|---|--|



Code Quality

This audit scope involves the smart contracts of the Umbrae project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring were recommended to optimize security measures.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the Umbrae project smart contract code in the form of GitHub access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Apart from libraries, its functions are used in external smart contract calls.

Severity Definitions

The severity levels assigned to findings represent a comprehensive evaluation of both their potential impact and the likelihood of occurrence within the system. These categorizations are established based on Hashlock's professional standards and expertise, incorporating both industry best practices and our discretion as security auditors. This ensures a tailored assessment that reflects the specific context and risk profile of each finding.

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies.
QA	Quality Assurance (QA) findings are informational and don't impact functionality. Supports clients improve the clarity, maintainability, or overall structure of the code.

Status Definitions

Each identified security finding is assigned a status that reflects its current stage of remediation or acknowledgment. The status provides clarity on the handling of the issue and ensures transparency in the auditing process. The statuses are as follows:

Significance	Description
Resolved	The identified vulnerability has been fully mitigated either through the implementation of the recommended solution proposed by Hashlock or through an alternative client-provided solution that demonstrably addresses the issue.
Acknowledged	The client has formally recognized the vulnerability but has chosen not to address it due to the high cost or complexity of remediation. This status is acceptable for medium and low-severity findings after internal review and agreement. However, all high-severity findings must be resolved without exception.
Unresolved	The finding remains neither remediated nor formally acknowledged by the client, leaving the vulnerability unaddressed.

Audit Findings

High

[H-01] BitMath#mostSignificantBit - Incorrect most significant bit calculation causes swap failures and potential loss of funds

Description

The `mostSignificantBit()` function in the `BitMath` library contains a critical bug where the return value `r` is incremented unconditionally instead of only when the corresponding condition is true. This causes the function to return incorrect values that are 254-255 positions too high, leading to incorrect bin ID calculations during swap operations when searching for the next active bin in the downward direction.

Vulnerability Details

The `mostSignificantBit()` function in `BitMath` is intended to find the index of the most significant bit in a 256-bit value. The function uses a binary search approach with conditional increments. However, the implementation has a critical flaw where all increments to `r` are executed unconditionally:

```
function mostSignificantBit(uint256 x) internal pure returns (uint8 r) {  
  
    if (x == 0) revert BitMath__ZeroInput();  
  
    unchecked {  
  
        if (x >= 0x100000000000000000000000000000000) x >>= 128;  
  
        r += 128; // Always executes, not conditional  
  
        if (x >= 0x10000000000000000) x >>= 64;  
  
        r += 64; // Always executes, not conditional  
  
        if (x >= 0x100000000) x >>= 32;  
  
        r += 32; // Always executes, not conditional  
  
        if (x >= 0x10000) x >>= 16;  
  
    }  
}
```



```

    r += 16;    // Always executes, not conditional

    if (x >= 0x100) x >>= 8;

    r += 8;     // Always executes, not conditional

    if (x >= 0x10) x >>= 4;

    r += 4;     // Always executes, not conditional

    if (x >= 0x4) x >>= 2;

    r += 2;     // Always executes, not conditional

    if (x >= 0x2) r += 1;

}
}

```

This bug causes `mostSignificantBit()` to return values that are always at least 254 too high (or 255 if the last condition is true). For example: input 1 (MSB at position 0) incorrectly returns 254.

The function is used in `_findNextActiveBinInWord()` when searching DOWN (when `swapForY` is false). When `mostSignificantBit()` returns an incorrect value, the calculated `nextBinId` will be incorrect.

This incorrect bin ID is then used in the swap logic in `_performSwap()`:

```

(uint24 nextBin, bool found) = _findNextActiveBin(currentBinId, swapForY);

if (!found) {
    revert LBPair__InsufficientLiquidityForSwap();
}

currentBinId = nextBin;

```

The incorrect bin ID could be invalid, point to a bin that doesn't have liquidity, causing the swap to fail or if the incorrect bin ID happens to be valid and has liquidity, the swap would execute against the wrong bin, causing incorrect pricing and potential loss of funds.

The bug affects all swaps that search in the downward direction (`swapForY = false`), which occurs when swapping Y for X tokens.

Impact

The bug causes incorrect bin ID calculations during downward bin searches, leading to swap failures for all swaps in the Y for X direction and potential loss of funds if an incorrect bin ID happens to be valid and contain liquidity, causing swaps to execute at incorrect prices.

Recommendation

Fix the `mostSignificantBit()` function by wrapping both the shift operation and the increment in braces so they execute together only when the condition is true. This matches the correct implementation from Uniswap V3's BitMath library from which it was adapted.

```
function mostSignificantBit(uint256 x) internal pure returns (uint8 r) {  
  
    if (x == 0) revert BitMath__ZeroInput();  
  
    unchecked {  
  
        if (x >= 0x100000000000000000000000000000000) {  
  
            x >>= 128;  
  
            r += 128;  
  
        }  
  
        if (x >= 0x10000000000000000) {  
  
            x >>= 64;  
  
            r += 64;  
  
        }  
  
    }  
}
```



```
    if (x >= 0x100000000) {  
        x >>= 32;  
        r += 32;  
    }  
  
    if (x >= 0x10000) {  
        x >>= 16;  
        r += 16;  
    }  
  
    if (x >= 0x100) {  
        x >>= 8;  
        r += 8;  
    }  
  
    if (x >= 0x10) {  
        x >>= 4;  
        r += 4;  
    }  
  
    if (x >= 0x4) {  
        x >>= 2;  
        r += 2;  
    }  
  
    if (x >= 0x2) r += 1;  
}  
}
```

Status

Resolved

[H-02] UmbraeLBFactoryV2#createPair - Missing activeId bounds validation allows permanent DoS of created pairs

Description

When creating a new liquidity pair through the factory, the `activeId` parameter is not validated against the protocol's defined bounds (`MIN_ACTIVE_BIN` and `MAX_ACTIVE_BIN`). An attacker can create pairs with extreme `activeId` values that cause any subsequent interaction to run out of gas, permanently bricking the pair.

Vulnerability Details

In `UmbraeLBFactoryV2.createPair()`, the factory validates token addresses and bin step, but accepts any `uint24` value for `activeId`:

```
function createPair(address tokenX, address tokenY, uint24 activeId, uint16 binStep)
    external override returns (address pair)
{
    // Validations

    if (tokenX == tokenY) revert LBFactory__IdenticalTokens();

    if (tokenX == address(0) || tokenY == address(0)) revert LBFactory__ZeroAddress();

    if (!_allowedBinSteps[binStep]) revert LBFactory__InvalidBinStep();

    // NO activeId validation against MIN_ACTIVE_BIN/MAX_ACTIVE_BIN

    // ... creates pair with arbitrary activeId
}
```

The `activeId` is passed directly to `UmbraeLBPairUpgradeable.initialize()` which stores it without validation:

```
function initialize(address tokenX_, address tokenY_, uint16 binStep_, uint24 activeId_)
    external initializer {

    // ...

    state.activeId = activeId_; // Stored without bounds check}
```

The root cause of the DoS is in `BinHelper.getPriceFromId()`, which uses an $O(n)$ loop where n equals the distance from `ACTIVE_BIN_ID` (8388608):

```
function getPriceFromId(uint24 binId, uint16 binStep) internal pure returns (uint256 price) {
    int256 exponent = int256(uint256(binId)) - int256(uint256(ACTIVE_BIN_ID));

    if (exponent > 0) {
        price = 1e18;

        for (uint256 i = 0; i < uint256(exponent); i++) { // O(n) iterations!
            price = (price * priceMultiplier) / BASIS_POINT_MAX;
        }
    } else {
        price = 1e18;

        for (uint256 i = 0; i < uint256(-exponent); i++) { // O(n) iterations!
            price = (price * BASIS_POINT_MAX) / priceMultiplier;
        }
    }
}
```

When `activeId = 0`, the exponent is `-8,388,608`, requiring 8.38 million loop iterations. At approximately 13 gas per iteration, this consumes ~109 million gas, far exceeding the 30 million block gas limit on Base.

This function is called by `getPrice()`, `mint()`, swap operations, and quote functions - meaning any interaction with the pair will revert with out-of-gas.

Impact

An attacker can permanently brick pair slots at minimal cost:

1. Call `createPair(tokenA, tokenB, 0, 25)` with `activeId = 0`
2. Pair is created and registered in the factory

3. Any user attempting to call `getPrice()`, `mint()`, `swap()`, or any function using `getPriceFromId()` runs out of gas
4. The pair slot for `(tokenA, tokenB, binStep=25)` is permanently burned
5. No valid pair can be created for this token combination with this bin step

The protocol already defines valid bounds in `SwapCalculator.sol` that are not enforced at pair creation:

```
uint24 internal constant MIN_ACTIVE_BIN = 4194304;    // 2^22
uint24 internal constant MAX_ACTIVE_BIN = 12582912;   // 2^23 + 2^22
```

Note: The test suite deliberately constrains fuzz testing to ± 50 bins from `ACTIVE_BIN_ID` (`BinHelper.t.sol:189`), masking this issue during normal testing.

Recommendation

Add bounds validation in `createPair()` using the existing constants:

```
// Add activeId bounds validation
if (activeId < SwapCalculator.MIN_ACTIVE_BIN || activeId > SwapCalculator.MAX_ACTIVE_BIN)
{
    revert LBFactory__InvalidActiveId();
}
```

Status

Resolved

[H-03] PositionNFTManager#claimFees - Incorrect per position fee tracking

Description

When multiple users have positions in the same pair and same bin, fee claiming does not attribute fees to each position proportionally. The first user to call `claimFees()` receives all fees accrued for that bin (for the entire LP balance held by the manager), and other users receive nothing for already-accrued fees, leading to fee theft between position owners.

Vulnerability Details

When `claimFees(tokenId, recipient, deadline)` is executed, the manager calls `pair.claimLPFees(pos.binIds[i], recipient)` for each bin in the position. The pair's `claimLPFees()` uses `msg.sender` as the LP holder. Since the caller is always `PositionNFTManager`, the pair sees a single LP holder per bin: the manager. So for a given bin, `balanceOf(PositionNFTManager, binId)` is the sum of all positions' LP in that bin, and `userDebt[PositionNFTManager][binId]` is a single shared debt.

As a result, `getClaimableFees(PositionNFTManager, binId)` returns the total claimable fees for the entire manager balance in that bin, not the share attributable to the specific position (`tokenId`). The first position owner to claim for that bin receives 100% of those accrued fees.

Impact

Users who share a bin with other position owners lose fees to the first claimer, resulting in direct loss of fee income.

Recommendation

Implement a similar per position fee tracking logic in `PositionNFTManager` as the pair's fee-per-share and debt logic so that each position (`tokenId`) only claims its proportional share of fees for each bin.

Status

Resolved

Medium

[M-01] UmbraeLBPairUpgradeable#compoundLPFees - Missing `notPaused` modifier allows adding liquidity to the pool when contract is paused

Description

The `compoundLPFees()` function modifies pool state by adding reserves and minting LP tokens, but lacks the `notPaused` modifier that is present on other liquidity-modifying functions like `mint()`. This allows users to add liquidity to the pool when the contract is paused, defeating the purpose of the pause mechanism.

Vulnerability Details

The `compoundLPFees()` function performs operations similar to `mint()` by adding liquidity to the pool:

```
function compoundLPFees(uint24 binId) external nonReentrant returns (uint256 lpTokensMinted) {  
    // ... fee calculation logic ...  
  
    // Update bin reserves with normalized amounts  
  
    bin.reserveX += compoundXNormalized.safe128();  
    bin.reserveY += compoundYNormalized.safe128();  
  
    // Mint LP tokens  
  
    _mint(msg.sender, binId, lpTokensMinted);  
  
    bin.totalShares += lpTokensMinted;  
  
    _addDebtForNewShares(msg.sender, binId, lpTokensMinted);  
}
```

However, unlike `mint()` which has the `notPaused` modifier, the `compoundLPFees()` function does not.

The pause mechanism is designed to freeze state changing operations during emergencies. When the contract is paused via `pause()` operations that modify pool



state should be blocked. However, `compoundLPFees()` can still execute and add liquidity to bins, bypassing the pause protection.

Impact

The pause mechanism fails to fully protect the contract state, as users can still add liquidity through fee compounding when the contract is paused, potentially allowing operations that should be blocked during emergency situations.

Recommendation

Add the `notPaused` modifier to the `compoundLPFees()` function to match the behavior of other liquidity modifying functions.

Status

Resolved

[M-02] UmbraeLBRouterUpgradeable#_getPriceFromOwnPair - Returns misleading 1e18 price when pair not found

Description

When querying a price for a pair that doesn't exist, the function returns 1e18 (1:1 ratio) instead of reverting. This dangerous default could cause integrations to make drastically incorrect valuations.

Vulnerability Details

In `UmbraeLBRouterUpgradeable.sol`, the `_getPriceFromOwnPair()` function iterates through all possible bin steps looking for a valid pair. If no pair is found for any bin step, it silently returns a 1:1 price ratio:

```
function _getPriceFromOwnPair(address tokenA, address tokenB) internal view returns
(uint256 price) {

    uint16[7] memory binSteps = [25, 10, 100, 50, 20, 5, 1];

    for (uint256 i = 0; i < 7; i++) {

        address pair = UmbraeLBFactoryUpgradeable(factory).getPair(tokenA, tokenB,
binSteps[i]);

        if (pair != address(0)) {

            // ... get price from pair

            return price;

        }

    }

    // DANGEROUS DEFAULT - no pair found for ANY binStep

    if (pair == address(0)) {

        // No pair exists, return 1:1 price

        return 1e18;

    }

}
```


This fallback value is then propagated through other functions causing the following issues:

1. `getCurrentMarketPrice()` calls `_getPriceFromOwnPair()` when no oracle is configured or when the oracle doesn't have a price. If no pair exists, it returns `1e18`, making it appear as if a valid 1:1 market price exists.
2. `getCurrentMarketBinId()` calls `getCurrentMarketPrice()` and then converts the price to a bin ID using `BinHelper.getIdFromPrice()`. When `1e18` is passed to `BinHelper.getIdFromPrice()`, the function has a special case that returns `ACTIVE_BIN_ID` (8388608):
3. `getBinsAroundMarketPrice()` uses `getCurrentMarketBinId()` to get the center bin and generates bins around it. When no pair exists, this results in bins being generated around bin ID 8388608.

For a WETH/USDC query where no Umbrae pair exists:

- Expected behavior: Revert with clear error
- Actual behavior: Returns `1e18` (1:1 ratio)
- Real market price: `~3000e18` (3000:1)
- Valuation error: 3000x incorrect

Impact

Integrations relying on this price feed could make drastically incorrect valuations when pairs don't exist. An integrator calling `getCurrentMarketPrice(WETH, USDC)` or any other affected function when no pair exists would receive a 1:1 price, valuing 1 WETH as 1 USDC instead of 3000 USDC - a 99.97% valuation error that could lead to significant financial losses in downstream applications.

Recommendation

Revert instead of returning a dangerous default value:

```
if (pair == address(0)) {
```



```
revert LBRouter__NoPairExists(tokenA, tokenB);  
  
}
```

Status

Resolved



[M-03] UmbraeLBPairUpgradeable#quoteSwap,quoteSwapIn - Incorrect volatility accumulator handling differs from `swap()` implementation

Description

The `quoteSwap()` and `quoteSwapIn()` functions incorrectly handle volatility accumulator updates compared to the actual `swap()` implementation. This leads to inaccurate fee calculations in quotes, particularly affecting `quoteSwapIn()` which is used in the router's `swapTokensForExactTokens()` function.

Vulnerability Details

The `swap()` function correctly handles volatility in two ways:

1. Before any fee calculation, `swap()` calls `_updateVolatility()` to apply time based decay.
2. In `_performSwap()`, volatility is only updated when specific conditions are met in `_shouldUpdateVolatility()`.

However, `quoteSwap()` and `quoteSwapIn()` have two differences:

1. Both quote functions start with `state.volatilityAccumulator` directly without applying time based decay.
2. Both quote functions update volatility unconditionally after moving to the next bin, without checking `_shouldUpdateVolatility()`.

This means quotes will use stale volatility values if time has passed since the last swap (missing decay), update volatility even when the filter period hasn't passed, update volatility even for dust swaps that don't meet the minimum threshold, and update volatility even when fees round to zero.

Impact

Quote functions return inaccurate fee calculations due to incorrect volatility handling:

- `quoteSwap()` returns less tokens as output than expected. It's not directly used in the router, but if used in frontend to calculate slippage parameters, it can set a lower `amountOutMin` for `swapExactTokensForTokens()`.

- `quoteSwapIn()` returns a larger `amountIn` than actually needed. It's used in `swapTokensForExactTokens()` internal calculations:

- If users set correct allowance or `amountInMax`, transactions will revert as the returned `amountIn` exceeds the actual requirement.

- If users are permissive with allowance and `amountInMax`, they will overprovide tokens and receive more output than desired, breaking the "exact tokens" functionality where users expect to provide the minimum required input for a desired output.

Recommendation

Update `quoteSwap()` and `quoteSwapIn()` to match the volatility handling logic in `swap()`:

1. Apply time-based volatility decay at the start of both functions by simulating `_updateVolatility()` logic
2. Add conditional volatility updates that respect `_shouldUpdateVolatility()` checks

Status

Resolved

[M-04] UmbraeLBPairUpgradeable#_calculateFeeForNetInput - Incorrect inverse fee calculation formula

Description

The `_calculateFeeForNetInput()` function uses an incorrect mathematical formula to calculate fees from net input amounts. This causes `quoteSwapIn()` to overestimate the required input amount, which breaks the exact out behavior of `swapTokensForExactTokens()` in the router, causing users to transfer more assets than necessary and receive more output tokens than specified.

Vulnerability Details

The fee calculation system uses two different formulas:

1. used in `_performSwap()`:

- Uses `FeeHelper.getFeeAmountFrom()` which implements: $\text{fee} = (\text{grossInput} * \text{feeRate}) / (10000 + \text{feeRate})$
- This correctly calculates fee FROM a gross input amount
- Net input after fee: $\text{netInput} = \text{grossInput} * 10000 / (10000 + \text{feeRate})$

2. used in `quoteSwapIn()` via `_calculateFeeForNetInput()`:

- Current implementation: $\text{fee} = \text{ceil}(\text{netInput} * \text{feeRate} / (10000 - \text{feeRate}))$
- This formula is mathematically incorrect

The correct inverse formula should be derived as follows:

- Given the calculation: $\text{netInput} = \text{grossInput} * 10000 / (10000 + \text{feeRate})$
- To find the inverse, we solve for `grossInput`:
 - Multiply both sides by $(10000 + \text{feeRate})$: $\text{netInput} * (10000 + \text{feeRate}) = \text{grossInput} * 10000$
 - Divide both sides by 10000 : $\text{grossInput} = \text{netInput} * (10000 + \text{feeRate}) / 10000$
- Since $\text{fee} = \text{grossInput} - \text{netInput}$, we substitute:

- $\text{fee} = \text{netInput} * (10000 + \text{feeRate}) / 10000 - \text{netInput}$
- Factor out `netInput`: $\text{fee} = \text{netInput} * ((10000 + \text{feeRate}) / 10000 - 1)$
- Simplify: $\text{fee} = \text{netInput} * (10000 + \text{feeRate} - 10000) / 10000$
- Therefore: $\text{fee} = \text{netInput} * \text{feeRate} / 10000$

The correct formula $\text{fee} = \text{netInput} * \text{feeRate} / 10000$ matches the pattern used in `FeeHelper.getFeeAmountTo()`, which calculates fees added to an amount using $(\text{amount} * \text{totalFee}) / \text{BASIS_POINTS}$. The only difference is that `_calculateFeeForNetInput()` should use ceiling division for safety in exact out swaps, resulting in $\text{fee} = (\text{netInput} * \text{feeRate} + 10000 - 1) / 10000$.

The current implementation uses $\text{fee} = \text{netInput} * \text{feeRate} / (10000 - \text{feeRate})$ which overestimates fees, especially at higher fee rates.

The impact propagates through the swap flow, `quoteSwapIn()` calls `_calculateFeeForNetInput()` for each bin's net input, and the incorrect formula overestimates fees, causing `totalFee` to be too high. `swapTokensForExactTokens()` uses `quoteSwapIn()` causing users to transfer the overestimated required input amount. Since the actual swap uses the correct forward formula, users receive more output than requested.

Impact

The incorrect fee formula causes `quoteSwapIn()` to overestimate required input amounts, breaking the exact out behavior of `swapTokensForExactTokens()` where users transfer more assets than necessary and receive more output tokens than the specified `amountOut` parameter, breaking the function's contract semantics.

Proof of Concept

Add the following fuzz test to `test/Issue063_QuoteSwapIn.t.sol`:

```
function testFuzz_quoteSwapIn_fee_calc_correctness(uint256 desiredOutput) public {
    // Bound to reasonable range
    desiredOutput = bound(desiredOutput, 1e15, 100e18);
```

```

_addLiquidityToActiveBin(1000e18);

(uint256 requiredInput,,,uint256 totalFee) = pair.quoteSwapIn(true, desiredOutput);

if (requiredInput == type(uint256).max) {

    // Insufficient liquidity - skip

    return;

}

// Do actual swap

address tokenX = pair.tokenX();

MockToken063(tokenX).mint(swapper, requiredInput);

vm.startPrank(swapper);

MockToken063(tokenX).transfer(address(pair), requiredInput);

vm.recordLogs();

uint256 actualOutput = pair.swap(true, swapper);

Vm.Log[] memory logs = vm.getRecordedLogs();

vm.stopPrank();

// Extract SwapDetailed event from logs

                                bytes32                swapDetailedTopic                =
keccak256("SwapDetailed(address,address,uint256,uint256,uint24,uint24,uint256,uint256,uint
256,uint256,bool)");

uint256 eventTotalFee;

for (uint256 i = 0; i < logs.length; i++) {

    if (logs[i].topics[0] == swapDetailedTopic) {

        ( , , , ,

            , eventTotalFee, , , ) =

            abi.decode(logs[i].data, (uint256, uint256, uint24, uint24, uint256,
uint256, uint256, uint256, bool));

        break;
    }
}

```

```

    }

    }

    vm.stopPrank();

    // 2 wei round tolerance

    assertApproxEqAbs(actualOutput, desiredOutput, 2, "exact output");

    assertApproxEqAbs(totalFee, eventTotalFee, 2, "exact fee estimate");

}

```

The test currently fails as the `actualOutput` and `totalFees` are way bigger than the desired output and the actual total fee. After applying the fix, the test should pass.

Recommendation

Update `_calculateFeeForNetInput()` to use the correct inverse formula by applying the following diff:

```

- uint256 denominator = 10000 - totalFeeBps;

- fee = (netInput * totalFeeBps + denominator - 1) / denominator;

+ fee = (netInput * totalFeeBps + 10000 - 1) / 10000;

```

Status

Resolved

[M-05] BinHelper#getPriceFromId - Precision loss and gas inefficiency due to loop-based exponentiation

Description

The `BinHelper.getPriceFromId()` function uses a simple loop based approach for exponentiation that accumulates rounding errors with each iteration and is gas inefficient compared to industry standard fixed-point math libraries. The implementation performs $O(n)$ iterations where n equals the exponent value, causing precision loss through integer division truncation and significant gas costs for bins far from the base price bin.

Vulnerability Details

The `getPriceFromId()` function calculates prices using the formula $(1 + \text{binStep}/\text{BASIS_POINT_MAX})^{\text{exponent}}$. The current implementation uses a loop approach:

```
function getPriceFromId(uint24 binId, uint16 binStep) internal pure returns (uint256 price) {
    int256 exponent = int256(uint256(binId)) - int256(uint256(ACTIVE_BIN_ID));

    if (exponent == 0) {
        return 1e18; // Base price
    }

    uint256 priceMultiplier = BASIS_POINT_MAX + binStep;

    if (exponent > 0) {
        price = 1e18;

        for (uint256 i = 0; i < uint256(exponent); i++) {
            price = (price * priceMultiplier) / BASIS_POINT_MAX;
        }
    } else {
        price = 1e18;

        for (uint256 i = 0; i < uint256(-exponent); i++) {
```

```

        price = (price * BASIS_POINT_MAX) / priceMultiplier;
    }
}
}

```

Each iteration performs integer division $(\text{price} * \text{priceMultiplier}) / \text{BASIS_POINT_MAX}$, which truncates the result, causing rounding errors that compound with larger exponents. Additionally, the loop-based approach has $O(n)$ complexity, requiring $|\text{exponent}|$ iterations. For an exponent of 100, the current implementation consumes approximately 12,799 gas, while a binary exponentiation approach ($O(\log n)$) consumes only 1,978 gas - a 84% reduction that becomes even more significant for larger exponents.

The implementation includes comments stating "Approximate calculation - production would use precise math library" and "Simplified for gas efficiency - actual implementation would use logarithmic math", indicating the team is aware this is not production ready.

Industry standard implementations use fixed-point math libraries with binary exponentiation ($O(\log n)$) that maintain precision throughout calculations and are significantly more gas efficient.

Impact

The precision loss accumulates with larger exponents, leading to incorrect price calculations for bins far from the active bin, which can result in incorrect swap prices, inaccurate liquidity calculations, and potential arbitrage opportunities. Additionally, the gas inefficiency makes the protocol more expensive to use, especially for swaps that traverse multiple bins.

Proof of Concept

Install PRBMath

```
forge install PaulRBerg/prb-math@release-v4
```

Create the following test file `test/libraries/BinHelperPrecisionLoss.t.sol`:



```

// SPDX-License-Identifier: MIT

pragma solidity ^0.8.26;

import "forge-std/Test.sol";

import "../src/libraries/BinHelper.sol";

import {UD60x18, ud} from "@prb/math/UD60x18.sol";

/**
 * @title BinHelperPrecisionLoss
 * @notice Minimal test demonstrating precision loss in BinHelper.getPriceFromId()
 * @dev Run: forge test --match-test testPrecisionLoss -vv
 */
contract BinHelperPrecisionLoss is Test {
    uint24 constant ACTIVE_BIN_ID = 8388608;

    uint256 constant BASIS_POINT_MAX = 10_000;

    /// @notice PRBMath implementation (reference)

    function getPriceFromIdPRBMath(uint24 binId, uint16 binStep) internal pure returns
(uint256) {
        int256 exponent = int256(uint256(binId)) - int256(uint256(ACTIVE_BIN_ID));

        if (exponent == 0) return 1e18;

        UD60x18 base = ud((uint256(BASIS_POINT_MAX + binStep) * 1e18) / BASIS_POINT_MAX);

        if (exponent > 0) {
            return base.powu(uint256(exponent)).unwrap();
        } else {
            UD60x18 powResult = base.powu(uint256(-exponent));

            UD60x18 one = ud(1e18);

            return one.mul(one).div(powResult).unwrap();
        }
    }
}

```

```

function testPrecisionLoss() public pure {

    uint16 binStep = 10; // 0.1%

    console.log("\n=== Precision Loss: Current vs PRBMath ===");

    console.log("Bin Step: 10 bps | Formula: (1.001)^exponent");

    console.log("");

    uint256[] memory exponents = new uint256[](5);

    exponents[0] = 100;

    exponents[1] = 500;

    exponents[2] = 1000;

    exponents[3] = 2000;

    exponents[4] = 5000;

    for (uint256 i = 0; i < exponents.length; i++) {

        uint256 exp = exponents[i];

        uint24 binId = uint24(ACTIVE_BIN_ID + exp);

        uint256 current = BinHelper.getPriceFromId(binId, binStep);

        uint256 prbMath = getPriceFromIdPRBMath(binId, binStep);

        uint256 diff = current > prbMath ? current - prbMath : prbMath - current;

        console.log("Exponent:", exp);

        console.log("  Current:", current);

        console.log("  PRBMath:", prbMath);

        console.log("  Difference:", diff, "wei");

        console.log("");

    }

}

```

Recommendation

Replace the loop-based implementation with a fixed-point math library like PRBMath. The recommended solution uses binary exponentiation ($O(\log n)$) with fixed-point arithmetic.

Status

Resolved



[M-06] **PositionNFTManager#createPositionWithStrategy, increasePositionWithStrategy** - Missing active bin ID slippage protection

Description

The `createPositionWithStrategy()` and `increasePositionWithStrategy()` functions in `PositionNFTManager` retrieve the active bin ID to calculate liquidity distribution but lack slippage protection against active bin movement or manipulation.

Vulnerability Details

Both `createPositionWithStrategy()` and `increasePositionWithStrategy()` call `IUmbracleBPair(pair).getActiveId()` to obtain the current active bin ID, which is then used to calculate the price from the active bin ID using `BinHelper.getPriceFromId()`, determine how to distribute liquidity around the active bin based on the selected strategy and mint liquidity to the active bin and surrounding bins. However, there is no validation to ensure that the active bin ID matches an expected value or hasn't moved beyond an acceptable threshold.

Impact

Users calling `createPositionWithStrategy()` or `increasePositionWithStrategy()` are vulnerable to active bin movement and manipulation, resulting in suboptimal liquidity distribution.

Recommendation

Add an active bin id slippage protection, similar to the `activeIdDesired` and `idSlippage` parameters in the router's `addLiquidity()` function.

Status

Resolved

[M-07] PositionNFTManager#claimFees - Single zero fee bin blocks all fee claims

Description

When `claimFees()` iterates over multiple bins in a position, if any bin has zero claimable fees, the entire transaction reverts due to `UmbraeLBPairUpgradeable.claimLPFees()` reverting on zero fees. This prevents users from claiming fees from bins that do have accumulated fees.

Vulnerability Details

In `PositionNFTManager.claimFees()`, the function iterates over all bins in a position and calls `pair.claimLPFees()` for each bin. The `claimLPFees()` function checks if there are claimable fees and reverts if both amounts are zero.

When a position spans multiple bins, it's common for some bins to have accumulated fees while others have none. The current implementation causes the entire fee claiming operation to fail if any bin in the loop has zero fees, even though other bins may have significant claimable fees.

Impact

Users with positions across multiple bins cannot claim fees from bins that have accumulated fees if any other bin in their position has zero fees. Since `PositionNFTManager` does not provide a function to claim fees for a specific bin, users are forced to either wait indefinitely until all bins accumulate fees or decrease or close their entire position (which claims fees internally as part of `burn()`).

Recommendation

Check if fees exist before calling `claimLPFees()` by using `pair.getClaimableFees()` first, and only call `claimLPFees()` if fees are available. Alternatively, modify the loop to handle `claimLPFees()` reverts gracefully.

Also, consider adding a function to allow users to claim fees for a specific set of bins only as it could be a more gas efficient alternative in case only a small amount of bins have fees or in case the user has a big position with many bins.

Status

Resolved



Low

[L-01] UmbraeLBFactoryV2#createPair - activeId is subject to frontrunning

Description

The `createPair()` function accepts `activeId` as a user provided parameter, allowing other users to frontrun pair creation transactions to set different initial prices which impacts liquidity provision and the first swaps on the pair.

Vulnerability Details

The `createPair()` function accepts `activeId` directly from the caller which determines the initial price of the pair and also affects the initial volatility calculations.

This allows other legit users and attackers to frontrun it with their own transaction using a different `activeId` to set an incorrect initial price that doesn't reflect the actual market price, leading to initial liquidity providers adding liquidity at incorrect prices, first swaps executing at prices far from market value, or temporary price manipulation until arbitrage corrects it.

Impact

Can mislead users during initial liquidity provision and affect the first swaps on newly created pairs.

Recommendation

Document this behavior clearly so users are aware that initial prices set during pair creation cannot be trusted and may not reflect actual market prices.

Status

Acknowledged

[L-02] UmbraeLBPairBeacon#scheduleUpgrade - Incorrect validation when emergency paused

Description

When the beacon is in emergency paused state, the validation in `scheduleUpgrade()` compares the new implementation against the paused implementation instead of the actual current implementation, which could allow scheduling invalid upgrades.

Vulnerability Details

The `scheduleUpgrade()` function validates that the new implementation is different from the current implementation. However, the `implementation()` function is overridden to return different values based on the pause state:

```
// In scheduleUpgrade() function:

if (newImplementation == implementation()) {
    revert SameImplementation();
}

// In implementation() function:

function implementation() public view override returns (address) {

    if (_emergencyPaused) {

        return _pausedImplementation;

    }

    return super.implementation();
}
```

When `_emergencyPaused` is `true`, `implementation()` returns `_pausedImplementation` instead of the actual current implementation stored in the parent `UpgradeableBeacon` contract. If the protocol is paused and an owner attempts to schedule an upgrade to the actual current implementation (the one that will be active when unpaused), the validation will compare against `_pausedImplementation` instead, potentially allowing an invalid upgrade to be scheduled.

Impact

The validation logic is incorrect when the beacon is in paused state, potentially allowing scheduling upgrades to the same implementation that's actually in use.

Recommendation

Modify the validation in `scheduleUpgrade()` to compare against `super.implementation()` instead of `implementation()` to ensure it always validates against the actual current implementation regardless of pause state.

Status

Acknowledged

[L-03] UmbraeLBFactoryV2#isPairVerified - Reverts on invalid input instead of returning false

Description

The `isPairVerified()` view function reverts when passed an address that is not a valid LBPair contract, instead of gracefully returning `false`. This breaks integrator expectations for boolean query functions and forces all callers to implement defensive error handling.

Vulnerability Details

In `UmbraeLBFactoryV2.sol`, the `isPairVerified()` function attempts to call external functions on the provided address without try-catch protection:

```
function isPairVerified(address pair) external view override returns (bool verified) {  
    if (pair == address(0)) return false;  
  
    UmbraeLBPairUpgradeable lbPair = UmbraeLBPairUpgradeable(pair);  
  
    address tokenX = lbPair.tokenX(); // Reverts if pair is not valid LBPair  
    address tokenY = lbPair.tokenY(); // Same issue  
  
    return _tokenTiers[tokenX] != IUmbraeLBFactory.VerificationTier.UNVERIFIED  
        && _tokenTiers[tokenY] != IUmbraeLBFactory.VerificationTier.UNVERIFIED;  
}
```

When `pair` is an EOA or a contract without the `tokenX()` function, the external call reverts with no function selector match, causing the entire `isPairVerified()` call to revert.

The same contract implements `getPairsMetadata()` which properly handles invalid addresses using try-catch:

```
// UmbraeLBFactoryV2.sol:277-302  
  
function getPairsMetadata(address[] calldata pairAddresses)  
    external view returns (PairMetadata[] memory metadata)
```

```

{
    for (uint256 i = 0; i < pairAddresses.length; i++) {
        UmbraeLBPairUpgradeable pair = UmbraeLBPairUpgradeable(pairAddresses[i]);

        try pair.tokenX() returns (address tokenX) {
            // ... build metadata
        } catch {
            // Skip invalid pairs - graceful handling

            continue;
        }
    }
}

```

This proves the defensive pattern is known and was applied elsewhere in the same contract, but was missed for `isPairVerified()`.

Impact

A function with signature `returns (bool verified)` implies it safely handles any address input without reverting, similar to how `address.isContract()` returns false for EOAs instead of reverting.

Recommendation

Implement defensive try-catch for external calls to gracefully handle invalid addresses. This matches the pattern already used in `getPairsMetadata()` and ensures consistent defensive programming throughout the contract.

```

function isPairVerified(address pair) external view override returns (bool verified) {
    if (pair == address(0)) return false;

    if (pair.code.length == 0) return false; // EOA check

    try UmbraeLBPairUpgradeable(pair).tokenX() returns (address tokenX) {

        try UmbraeLBPairUpgradeable(pair).tokenY() returns (address tokenY) {

```

```
        return _tokenTiers[tokenX] != IUmbraeLBFactory.VerificationTier.UNVERIFIED
            && _tokenTiers[tokenY] != IUmbraeLBFactory.VerificationTier.UNVERIFIED;
    } catch {
        return false;
    }
} catch {
    return false;
}
}
```

Status

Resolved

[L-04] UmbraeLBRouterUpgradeable#_getPriceFromOwnPair() - Hardcoded binSteps list prevents finding pairs with newly added binSteps

Description

The `_getPriceFromOwnPair()` function uses a hardcoded list of binSteps, preventing it from finding pairs created with binSteps that are added to the factory after deployment via `setAllowedBinStep()`.

Vulnerability Details

The `_getPriceFromOwnPair()` function in `UmbraeLBRouterUpgradeable` contains a hardcoded array of binSteps that uses to find an existing pair. However, the factory contract allows the owner to dynamically add new binSteps via the `setAllowedBinStep()` function. If the factory owner adds a new binStep, the `_getPriceFromOwnPair()` function will not find pairs created with this new binStep because it only checks the hardcoded list.

Impact

Pairs created with newly added binSteps cannot be discovered by `_getPriceFromOwnPair()`, causing the function to return misleading fallback prices instead of actual market prices.

Recommendation

To maintain a preference order, first check the hardcoded list for common binSteps (for gas efficiency) as currently used, if not found, then fallback to call `factory.getAllPairs(tokenA, tokenB)` to retrieve all pairs for the token pair to discover pairs with newly added binSteps.

Status

Resolved

[L-05] UmbraeLBPairUpgradeable#mint - Invalid bin ID validation missing

Description

The `mint()` function does not validate that provided bin IDs are within the valid trading range (`MIN_ACTIVE_BIN` to `MAX_ACTIVE_BIN`), allowing users to add liquidity to bins that will never be used in swaps. These invalid bins are added to the active bins list and pollute the data structure.

Vulnerability Details

The `mint()` function accepts an array of bin IDs without validating they are within the practical trading bounds defined by `MIN_ACTIVE_BIN` and `MAX_ACTIVE_BIN`.

When `mint()` is called with invalid bin IDs, the function calls `_markBinActive()` for bins with liquidity and adds the invalid bin ID to `activeBinsList` and sets the bitmap bit. However, swaps will never use these bins because `_findNextActiveBin()` filters them out. When liquidity is removed via `burn()`, the bin is marked inactive in the bitmap but remains in `activeBinsList`.

Impact

Users can add liquidity to bins outside the valid trading range, which will never be used in swaps, creating "dead" liquidity. These invalid bins pollute the `activeBinsList` data structure, affecting `getActiveBins()` and related functions.

Recommendation

Add validation in `mint()` to reject bin IDs outside the valid trading range.

Status

Acknowledged

[L-06] PositionNFTManager#_calculateSpotDistribution, _calculateCurve Distribution, _calculateBidAskDistribution - Bin ID validation uses full uint24 range instead of protocol bounds

Description

Bin ID range checks in `PositionNFTManager` use `0` and `type(uint24).max` instead of the protocol defined bounds `MIN_ACTIVE_BIN` and `MAX_ACTIVE_BIN` from `SwapCalculator`, making the validation looser than the rest of the system and allowing computed bin IDs that are outside the intended trading range.

Vulnerability Details

The protocol defines the allowed active bin range in `SwapCalculator` that is enforced when creating pairs in `UmbraeLBFactoryV2.createPair()` and when updating the active bin in `UmbraeLBPairUpgradeable` after swaps. In `PositionNFTManager`, the code validates that each computed bin ID fits in a `uint24` with `binIdInt >= 0 && binIdInt <= int256(uint256(type(uint24).max))`

This allows any bin ID from `0` to `16_777_215`, while the protocol only considers bins in `[4_194_304, 12_582_912]` as valid for trading. So positions could theoretically be created or increased with bins outside the protocol bounds, leading to liquidity in bins that the pair and swap logic treat as out of bounds.

Impact

Positions can be created or increased with bin IDs outside the protocol's intended range, so liquidity may sit in bins that are never used by the pair.

Recommendation

Replace the current validation checks with bounds that use `SwapCalculator.MIN_ACTIVE_BIN` and `SwapCalculator.MAX_ACTIVE_BIN`.

Status

Resolved

[L-07] PositionNFTManager#increasePositionWithStrategy - positions created without strategy use default strategy values

Description

When a position is created using `createPosition()` (without strategy), the `strategy`, `binRange`, and `bidAskRatio` fields remain at their default values (0). If `increasePositionWithStrategy()` is subsequently called on such a position, it will read these default values and treat the position as having Spot strategy with `binRange = 0`, causing all liquidity to be added only to the current active bin, which may differ from the user's original position bins.

Vulnerability Details

The `createPosition()` function allows users to manually specify bins and liquidity amounts without using any strategy. When storing the position the strategy-related fields are not initialized.

When `increasePositionWithStrategy()` is called on such a position, it reads these uninitialized values. With `binRange = 0` and `strategy = Strategy.Spot` (as Spot is in the default enum value), the calculation logic adds all liquidity to the current active bin, which may be different from the bins in the user's original position.

Impact

Users who create positions without a strategy and then call `increasePositionWithStrategy()` will have liquidity added to the current active bin instead of their original position bins, leading to unexpected liquidity distribution.

Recommendation

Add a validation check in `increasePositionWithStrategy()` that verifies `position.binRange > 0`, since valid strategies require a non zero `binRange`. Alternatively, consider adding a `NoStrategy` enum value to the `Strategy` enum as default value and use it as the validation check.

Status

Resolved



QA

[Q-01] BinHelper#ACTIVE_BIN_ID - Misleading constant name

Description

The constant `ACTIVE_BIN_ID` has a misleading name and incorrect documentation comment. The constant represents a reference point for 1:1 price calculations (value $8388608 = 2^{23}$), not the actual current active bin ID.

Vulnerability Details

The `BinHelper` library defines a constant `ACTIVE_BIN_ID` that is used as a reference point in price calculations where when `binId == ACTIVE_BIN_ID`, the price equals 1e18 (1:1 ratio).

However, the actual current active bin ID is stored in `UmbraeLBPairUpgradeable` as `state.activeId` and can be retrieved via `getActiveId()`. The `state.activeId` value changes dynamically as swaps occur, while `ACTIVE_BIN_ID` is a compile-time constant that never changes.

The comment on the constant states "The active bin is the current price bin", which incorrectly suggests this constant represents the current active bin. This is misleading because the constant never changes and is only used for price calculations relative to a 1:1 price point, not to represent the current trading bin.

Impact

Misleading naming and documentation can cause confusion and make the codebase harder to understand and maintain.

Recommendation

Rename `ACTIVE_BIN_ID` to `BASE_PRICE_BIN_ID` or `REFERENCE_BIN_ID` to accurately reflect its purpose as a reference point for 1:1 price calculations. Update the documentation comment to clarify that this is the reference bin ID used for price calculations where the price equals 1:1 (1e18).

Status

Acknowledged

[Q-02] Multiple contracts - Floating pragma**Description**

Multiple contracts use floating pragma statements `pragma solidity ^0.8.26`, instead of fixed version pragmas.

Vulnerability Details

Floating pragmas allow contracts to be compiled with different compiler versions than those tested, potentially introducing unidentified security issues.

Impact

Different pragma versions in test and production may pose unidentified security issues.

Recommendation

Consider locking the pragma version to the specific version `pragma solidity 0.8.26`.

Status

Acknowledged

[Q-03] Multiple contracts - Unused error definitions

Description

Multiple custom error definitions exist in the codebase but are never used

Vulnerability Details

Multiple custom errors are defined across multiple contracts and libraries but are never used in revert statements:

- UniswapV3PriceOracle.sol: Oracle__InsufficientHistory
- IUmbraeLBPair.sol: LBPair__BinReserveOverflow
- BinManager.sol:
 - BinManager__InsufficientLiquidity
 - BinManager__BinEmpty
- FlashLoanHelper.sol:
 - FlashLoanHelper__ExceedsBalance
 - FlashLoanHelper__InsufficientRepayment
 - FlashLoanHelper__InvalidFee
 - FlashLoanHelper__CallbackFailed
- UmbraeLBPairUpgradeable.sol:
 - LBPair__FlashLoanInProgress
- UmbraeLBRouterUpgradeable.sol:
 - LBRouter__InvalidBinStep
 - LBRouter__Unauthorized

Impact

Unused error definitions add unnecessary code and may mislead developers about available error handling mechanisms.

Recommendation

Remove the unused error definitions.

Status

Acknowledged



[Q-04] UmbraeLBRouterUpgradeable - Dead code functions

Description

Two internal functions are defined but never called in the codebase, increasing contract complexity unnecessarily.

Vulnerability Details

`_calculateFee()` and `_calculateSwapOutput()` are defined in `UmbraeLBRouterUpgradeable` but never invoked. Both functions appear to be remnants from previous implementations that were refactored.

Impact

Dead code adds unnecessary complexity to the contract.

Recommendation

Remove the dead code functions `_calculateFee()` and `_calculateSwapOutput()` from `UmbraeLBRouterUpgradeable`.

Status

Acknowledged

[Q-05] **UmbraeLBPairUpgradeable#setFeeParameters** - **Missing event emission**

Description

The `setFeeParameters()` function updates fee parameters but does not emit an event, making it difficult to track state changes off-chain.

Vulnerability Details

The `setFeeParameters()` function in `UmbraeLBPairUpgradeable` allows the factory to update fee parameters. However, unlike other admin functions such as `pause()` and `unpause()` which emit events, this function does not emit any event when the parameters are changed.

Impact

Missing event emission reduces transparency and makes it difficult for off-chain systems to monitor and index fee parameter changes.

Recommendation

Add an event emission in `setFeeParameters()` to log the new fee parameters.

Status

Acknowledged

[Q-06] PositionNFTManager#_calculateBidAskDistribution - bidAskRatio not unused

Description

The natspec and public API document `bidAskRatio` as controlling the split of liquidity value between bid and ask sides (e.g. 70% bid / 30% ask), but the parameter is never used in `_calculateBidAskDistribution()`.

Vulnerability Details

`_calculateBidAskDistribution()` accepts `bidAskRatio` as its fifth parameter. The natspec states:

- "bidAskRatio controls the split between bid and ask VALUE"
- "If bidAskRatio = 70: 70% of total value on bid side, 30% on ask side"

In the function signature the parameter is commented as reserved: `uint8 /* bidAskRatio - reserved for future weighting.`

The function body never reads `bidAskRatio`. Bid-side bins always receive `amountX` distributed evenly; ask-side bins always receive `amountY` distributed evenly. There is no logic that would shift value between bid and ask based on the ratio.

Impact

Integrators and users are misled by documentation into believing that `bidAskRatio` controls the bid/ask value split; the parameter has no effect on distribution, which can lead to wrong assumptions and integration bugs.

Recommendation

Update all natspec and interface documentation to state that bid/ask weighting is reserved for a future version.

Status

Resolved

Centralisation

The Umbrae project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.



Centralised

Decentralised

Conclusion

After Hashlock's analysis, the Umbrae project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au

Contact: info@hashlock.com.au



#hashlock.